

1й билет:

**Вопрос по JAVA: Понятия объекта и инкапсуляции. Понятие класса.
Объявление классов в Java. Члены класса. Доступ к членам класса.**

Объект – это самоописывающая структура данных обладающая внутреннем состоянием и способная обрабатывать передаваемое её сообщение

Если разбить на пункты:

1. Описывает сам себя
2. Имеет внутреннее состояние (Внутри могут быть другие объекты, переменные, одним словом поля)
3. Может обработать сообщение (Различные методы (метод - это способ передачи сообщения))

Понятие инкапсуляции – внутреннее состояние объекта должно быть доступно только в самом объекте и изменяться с помощью сообщений, те доступ к внутренностям только через сообщения

Официальное определение - **доступ к внутреннему состоянию(полям) объекта происходит через сообщения(методы)**

Класс - это шаблон для порождения объектов (по сути)

Другое определение класса - тип данных, значением которого являются объекты, имеющие схожее внутреннее состояние и обрабатывающие одинаковый набор сообщений.

Если разбить на пункты:

1. Тип данных (тип данных)
2. Значения – объекты с схожим внутренним состоянием (те же поля, может быть с другими значениями)
3. Значения обрабатывают одинаковый набор сообщений (методы одни и те же сохраняются)

Класс в java объявляется словом class

Пример объявления:

```
class A{ }
```

Перед словом класс может быть модификатор доступа

В каждом файле обязательно должен быть публичный класс и у него обязательно должно имя совпадать с именем класса.

Членами класса являются:

1. Экземплярные поля – хранят внутреннее состояние объекта. Именованная составная часть внутреннего состояния объекта.
2. Экземплярные методы – это подпрограмма выполняющая обработку передаваемого объекту сообщения. Имеет доступ к внутреннему состоянию объекта и передает сообщение.

3. Экземплярные конструкторы – инициализируют только что созданные объекты класса
4. Статические поля – хранят данные общие для всех объектов класса
5. Статические методы – выполняют действия для которых не нужен доступ к конкретному объекту класса.
6. Вложенные классы – объекты необходимые для реализации данного класса.
7. Статический конструктор – инициализирует статические поля класса(в java такого нет, его функцию выполняют статические блоки см. лекцию 4)

Доступ к членам класса зависит от их модификатора доступа.

Модификаторы:

1. Private – доступ только из тела класса
2. Без модификатора – доступ только класса или классов того же пакета
3. Protected – доступ из класса, классов из того же пакета и наследников класса
4. Public – доступ откуда угодно

Вопрос по C++: Понятие шаблона. Типовые формальные параметры шаблона. Нетиповые формальные параметры шаблона. Нетиповые формальные параметры шаблона

Шаблон – это обобщенный тип или функция.

Типовые формальные параметры шаблона – это параметры которые определяют типы данных с которыми будет работать шаблон(задаются typename или class)

Нетиповые формальные параметры шаблона – это параметры которые задают какие-то константы и значения, не связанные с типами.

Шаблонные формальные параметры шаблона – позволяет параметризовать шаблон другим шаблоном, такой шаблом синтаксически записывается как шаблон без тела

Шаблон объявляется словом template затем идут скобочки <> внутри которых перечисляются формальные параметры. Затем идет тело шаблона

Формальные параметры вообще – это идентификаторы которые видны только с теле шаблона и могут обозначать типы данных, значения, другие шаблоны

При применении шаблона, формальные параметры заменяются на конкретные типы данных и значения и шаблоны и код компилируется.

Билет 2:

Вопрос по Java: Экземплярные поля. Статические поля

1. Экземплярные поля – хранят внутреннее состояние объекта. Именованная составная часть внутреннего состояния объекта.
2. Статические поля – хранят данные общие для всех объектов класса

Вопрос по C++: Виртуальное наследование.

Виртуальное наследование – это способ реализации наследования, гарантирующий, что базовый класс не будет включён ни в один из производных классов более чем в одном экземпляре.

А теперь, нормальными буквами.

```
ct: 1      #include <iostream>
    2
    3      class A{
    4          public:
    5              int x = 10;
    6          virtual int f(){
    7              return 5;
    8          }
    9      };
   10
   11      class B : public A{
   12          public:
   13              int f(){
   14                  return 6;
   15              }
   16      };
   17
   18      class C : public A, B{
   19
   20      };
   21
```

Это без виртуального, при создании экземпляра класса C будет создаваться два поля x, что нам не особо нужно и не понятно к какому полю обратиться.

А вот код с виртуальным наследованием будет создаваться одно поле x

```
class A{
    public:
        int x = 10;
        virtual int f(){
            return 5;
        }
};

class B : public virtual A{
    public:
        int f(){
            return 6;
        }
};

class C : public virtual A, B{
};
```

Билет 3:

Вопрос по Java: Экземплярные и статические методы. Объявление методов в Java. Перегрузка методов. Виртуальные методы.

Экземплярные методы – это подпрограмма выполняющая обработку передаваемого объекту сообщения. Имеет доступ к внутреннему состоянию объекта и передает сообщение

Статические методы – Выполняют действия к которым не нужен доступ к конкретному объекту класса.

Модификатор доступа + тип возвращаемых данных + имя метода(сигнатура) {тело метода}

Модификаторы:

1. Private – доступ только из тела класса
2. Без модификатора – доступ только класса или классов того же пакета
3. Protected – доступ из класса, классов из того же пакета и наследников класса
4. Public – доступ откуда угодно

Если метод статичный, то перед модификатором пишут static

Перегрузка – объявление для одного класса нескольких методов с одинаковым именем и разными сигнатурами.

Сигнатура – это информация о количестве и типах формальных параметров метода.

Раннее связывание – определение адреса вызываемого экземплярного метода **во время компиляции программы**. Например, в с++.

Позднее связывание – определение адреса вызываемого экземплярного метода на основе информации о классе объекта **во время выполнения программы**. Например, в Java.

Экземплярные методы для которых выполняется позднее связывание называются виртуальными.

Все методы в java виртуальные.

Выводы:

1. Для всех экземплярных методов будет выполняться позднее связывание
2. Конкретная реализация метода будет определяться во время исполнения
3. Нет необходимости знать точно тип объекта для работы с ним через виртуальные методы – достаточно знать, что объект принадлежит классу или наследнику класса, в котором объявлен метод.

Вопрос по C++: Иерархия наследования и классы противоречия. Основная проблема противоречивых иерархий.

Иерархия наследования – это ориентированный ациклический (направленный и без циклов) граф, в котором если класс А базовый и В наследует его, то от А к В проводится дуга.

Класс противоречия – это класс в который из какого-то другого класса ведут как минимум 2 пути. Пример, тут класс A будет классом противоречия

```
2
3  class A{
4      public:
5          int x = 10;
6          virtual int f(){
7              return 5;
8          }
9      };
10
11  class B : public A{
12      public:
13          int f(){
14              return 6;
15          }
16      };
17
18  class C : public A, B{
19      };
20
21
```

Основной проблемой класса противоречий является многократное включение полей и методов базового класса в произвольный.

Билет 4.

Вопрос по Java: Понятие экземплярного конструктора. Конструктор по умолчанию. Объявление конструктора в Java.

Экземплярный конструктор – экземплярный метод, который инициализирует только что созданный объект.

Конструктор по умолчанию – экземплярный конструктор с пустой сигнатурой

В java конструктор объявляется в теле класса и имеет то же имя, что и класс. Ничего не возвращает, соответственно не нужно указывать тип возвращаемых данных.

Пример:

```
class A{
    A(){
        System.out.println("Создали");
    }
}
```

Вопрос по C++: Наследование от нескольких базовых классов. Противоречия в именах наследуемых членов класса. Разрешение противоречий в именах.

C++ поддерживает множественное наследование. Пусть класс A наследует классы AA, AAA, и AAAA, тогда это будет записываться как:

```
class A : AA, AAA, AAAA{}
```

Также есть возможность установить модификатор доступа для наследуемых классов, например:

```
class A : public AA, AAA, AAAA{}
```

При множественном наследовании может возникнуть проблема одинаковых имен в различных базовых классах. Такая проблема разрешается конструкцией, когда мы обращаемся к конкретному базовому классу от объекта. Например:



```
int main() {
    C CC;
    std::cout<<CC.A::x;
    return 0;
}
```

Билет 5

Вопрос по Java: Размещение объектов в памяти. Операция «new». Массивовые литералы.

В java объекты хранятся в динамической памяти. Объекты не могут лежать в локальных переменных и параметрах методов, в полях объектов и элементах массивов. Вместо этого там хранятся только указатели на объекты, которые в языке Java называются объектными ссылками. Прimitives типы могут лежать в локальных переменных и параметрах методов, в полях объектов и элементах массивов.

Создания нового объекта выполняется с помощью операции new.

Массивовые литералы – это создание массивов с помощью фигурных скобок, например

```
new int[]{1, 2, 3}
```

Вопрос по C++: Динамическое приведение типов.

Для динамического приведения в с++ используется оператор `dynamic_cast` он принимает на вход ссылку на объект и тип необходимой ссылки `T`. Если он находит в объекте тип `T` то возвращает ссылку на него, иначе нулевую ссылку.

```
int main() {  
    B BB;  
    A *AA = &BB;  
    B *BBB = dynamic_cast<B*>(AA);  
    std::cout<<BBB->f();  
    return 0;  
}
```

Билет 6.

Вопрос по Java: Понятие статического конструктора. «Static»-блоки.

Статический конструктор – инициализирует статические поля.

В java отсутствуют статические конструкторы, но зато существуют static блоки.

Статические блоки выполняются во время запуска программы, при первом запуске класса, ещё до того, как этот класс будет использоваться в программе, до создания его экземпляров, вызова статических методов и обращения к ним и тд.

Если объявление класса содержит несколько статических блоков, то они будут выполняться в том порядке, в котором они перечислены в теле класса.

Пример:

```
static{
    a = 190;
}
```

Вопрос по C++: Основной способ наследования в C++. Переопределение методов.

Одиночное наследование в с++ - БАЗА

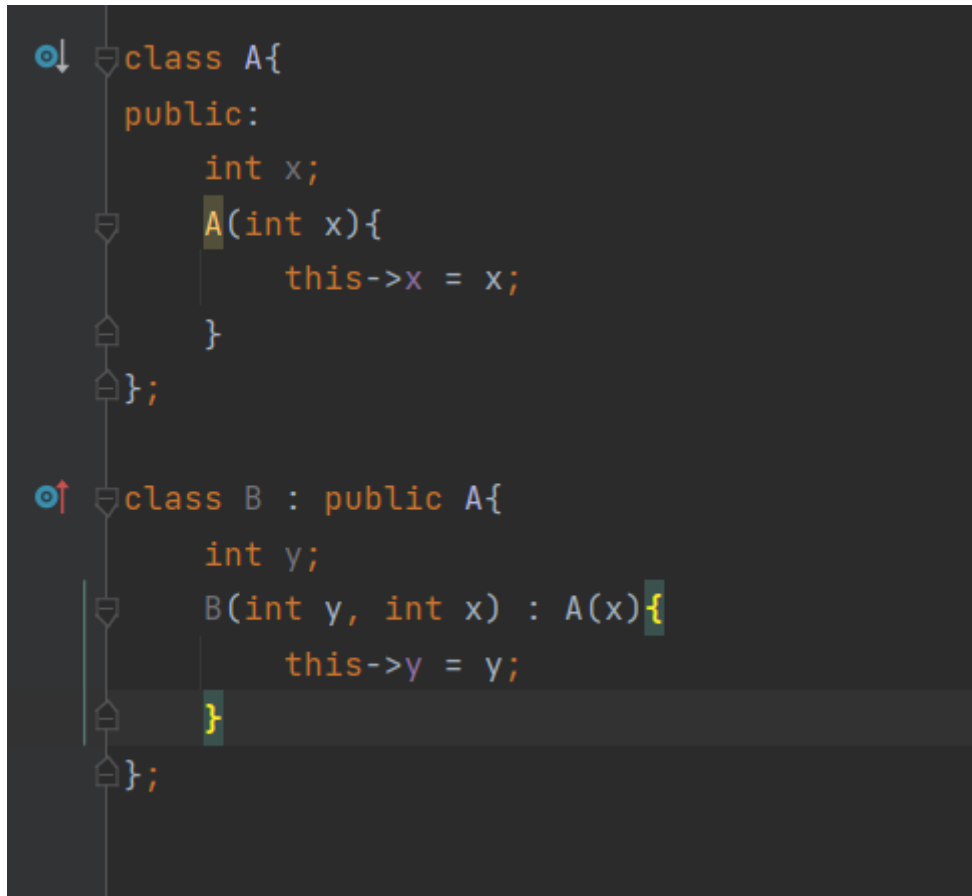
Вначале объявляется имя класса, потом, через двоеточие базовый класс

```
class A : B{}
```

Конструктор базового класса вызывается в конструкторе класса наследника, если он не был конструктором по умолчанию, иначе вызов будет автоматическим

Например:


```
A(int a, int b) : B(b){
}
```



```
class A{
    public:
        int x;
        A(int x){
            this->x = x;
        }
};

class B : public A{
    int y;
    B(int y, int x) : A(x){
        this->y = y;
    }
};
```

Билет 7.

Вопрос по Java: Понятие субтипизации. Явная и неявная субтипизация.

Определение 1: Тип данных A является подтипом(subtype) типа данных B, если программный код, рассчитанный на обработку значений типа B, может быть корректно использован для обработки значений типа A.

Определение 2: Субтипизация(Subtype) – свойство ЯП, означающее возможность использования подтипов в программах.

($\text{Int} \leq \text{Double}$)

$A \rightarrow B$ (A наследует B), т.е. приобретает поля класса B

Явная и неявная субтипизация

В Java используется явная субтипизация.

Зам 1. В ЯП, поддерживающих неявную типизацию, решение о том, является ли A подтипом типа B, происходит на основе структуры значений этих типов.

Зам 2. В ЯП, поддерживающих явную типизацию, тип A является подтипом B $\Leftrightarrow$ A extends B

($A \leq B$) $\Leftrightarrow$ (A extends B)

Отношения Агрегации ---- нет наследования(класс внутри класса)♦

Отношения Субтипизации ---- класс наследует поля другого класса(extends) ▲

Пример явной и неявной субтипизации

```
List<Integer> list = new ArrayList<>();  
list.add(1);  
list.add(2);  
list.add(3);  
  
// Явное приведение типа  
ArrayList<Integer> arrayList = (ArrayList<Integer>) list;
```

Вопрос по C++: Проблема копирования объектов. Объявление конструктора копий. Перегруженная операция присваивания.

Проблема копирования – по умолчанию создается побитовая копия внутреннего состояния объекта. Но есть ситуации когда это не работает хорошо, например у при копировании массива у нас будет указатель на один и тот же массив, что при вызове деструктора одного из объектов приведет к потере массива.

Для правильного копирования создается конструктор копий. По сути обычный конструктор в который мы передаем константный объект этого класса. Например:

```
class A{  
    int i;  
    A(const A &AA){  
        this->i = AA.i;  
    }  
};
```

Перегрузка операции присваивания, перегружаем операцию присваивания правильным образом, чтоб создавалась копия

```
    }  
    A& operator=(const A &AA){  
        this->i = AA.i;  
    }  
};
```

В примерах не решается проблема копирования, тк было лениво её решать. Но вы можете без проблем додумать сами её решение. На всякий случай скину из других источников

фото

Этот конструктор будет автоматически вызываться всякий раз, когда необходимо создать копию объекта.

```
IntArray::IntArray(const IntArray &obj): n(obj.n) {  
    a = new int [n];  
    std::copy(obj.a, obj.a + n, a);  
}
```

нужно

Также необходимо переопределить операцию присваивания

```
IntArray& IntArray::operator= (const IntArray &obj) {  
    if (this != &obj) {  
        int *new_a = new int [obj.n];  
        std::copy(obj.a, obj.a + obj.n, new_a);  
        delete [] a;  
        n = obj.n;  
        a = new_a;  
    }  
    return *this; }
```

нужно

Билет 8.

Вопрос по Java: Явная и неявная субтипизация. Вызов конструктора базового класса. Операция приведения типа. Переопределение метода.

Явная и неявная субтипизация

В Java используется явная субтипизация.

Зам 1. В ЯП, поддерживающих неявную типизацию, решение о том, является ли А подтипом типа В, происходит на основе структуры значений этих типов.

Зам 2. В ЯП, поддерживающих явную типизацию, тип А является подтипом В $\Leftrightarrow$ А extends В

$(A \leq B) \Leftrightarrow (A \text{ extends } B)$

Отношения Агрегации ---- нет наследования(класс внутри класса)◆

Отношения Субтипизации ---- класс наследует поля другого класса(extends)▲

Пример явной и неявной субтипизации

```
List<Integer> list = new ArrayList<>();  
list.add(1);  
list.add(2);  
list.add(3);  
  
// Явное приведение типа  
ArrayList<Integer> arrayList = (ArrayList<Integer>) list;
```

Конструктор базового класса(класса от которого наследуем) вызывается оператором `super()`, если конструктор базового класса был дефолтным, то это происходит автоматически.

Пусть мы хотим привести объект типа А к типу В. Если тип В является одним из типов объекта типа А, тогда возвращается нужный объект типа В, иначе порождается исключение. В Java динамическое приведение записывается как `(тип)obj`

Переопределение метода - это замена тела экземплярного метода базового класса в производном классе, позволяющая объекту производного класса по-другому обрабатывать то же самое сообщение.

Вопрос по C++: Объекты в глобальной памяти. Объекты в полях других объектов.

Если объект размещается в глобальной переменной или в статическом поле класса, то конструктор для него вызывается до передачи управления в функцию `main`, а деструктор – после завершения функции `main`.

Если объект находится в теле другого класса, то его конструктор можно вызвать из конструктора другого класса таким образом:

```
class B{
public:
    B(int ballIU9){
        std::cout<<ballIU9;
    }
};

class A{
    B BB;
    A(int i) : BB( ballIU9: i){}
};
```

Если мы захотим в данном примере удалить класс А, то автоматически вызовется деструктор и объекта класса В

Билет 9.

Вопрос по Java: Абстрактные классы.

Абстрактные классы – это классы в которых содержатся абстрактные методы. Класс который наследует абстрактный класс обязан переопределить все методы абстрактного класса. Экземпляр абстрактного класса нельзя создать. Пример

```

1  abstract class A{
    1 implementation
2  abstract int ballIU9();
3  }
4
5  class ABC extends A{
6  int ballIU9(){
7      System.out.println("IU");
8      return 0;
9  }
10 }
11

```

Вопрос по C++: Объявление деструктора. Объекты в автоматической памяти. Автоматический вызов деструктора.

Деструктор – это экземплярный метод, предназначенный для освобождения ресурсов, принадлежащих объекту, непосредственно перед уничтожением этого объекта.

Пример:

```

~A(){
}

```

В классе могут быть ссылки и различные данные которые при удалении объекта приводят к утечке данных. В языках без сборки мусора нужно прописывать деструктор, который освобождает ресурсы.

Автоматическая переменная (она же переменная в автоматической памяти и только локальные переменные могут быть автоматическими). Память под автоматическую переменную выделяется автоматически всякий раз, когда исполнение кода достигает определения этой переменной

Объекты в автоматической памяти – создаются внутри блока кода и когда блок кода завершается автоматически вызывается деструктор этих объектов.

Пример таких объектов

```
17
18     void F(){
19         int a;
20         int b;
21         A AA;
22     }
23
```

Билет 10.

Вопрос по Java: Объявление интерфейсов. Реализация интерфейсов. Наследование интерфейсов

Интерфейс – это тип данных, представляющий собой набор абстрактных методов.

```
1 interface A{
2     int ballIU9();
3     int ballIU8();
4     int ballIU6();
5 }
6
```

В интерфейсах не нужно прописывать модификаторы методов. Подтипы интерфейсов обязаны определить все методы интерфейса, если подтипом является абстрактный класс, то ему не нужно определять методы.

Реализацией интерфейсов является наследование классом интерфейса. Когда наследуем интерфейс, то мы обязаны реализовать все его методы. Можно наследовать несколько интерфейсов через запятую после ключевого слова `implements`

```
1 interface A{
2     int ballIU9();
3     int ballIU8();
4 }
5
6 interface B{
7     int ballIU7();
8     int ballIU6();
9 }
10
11 class AB implements A, B{
12     public int ballIU9(){
13         System.out.println("IU");
14         return 0;
15     }
16     public int ballIU8(){
17         System.out.println("IU");
18         return 0;
19     }
20     public int ballIU7(){
21         System.out.println("IU");
22         return 0;
23     }
24     public int ballIU6(){
25         System.out.println("IU");
26     }
27 }
28
```

Вопрос по C++: Создание объектов в динамической памяти. Создание массивов в динамической памяти. Удаление объектов и массивов.

Динамическое выделение памяти под значение любого типа выполняется в C++ операцией `new`. Она совмещает выделение памяти с инициализацией (с вызовом конструктора) и возвращает указатель на созданный и инициализированный объект. Примеры:

```
1  #include <iostream>
2
3  class A{
4  public:
5      int a, b;
6      A(){}
7      A(int a, int b){
8          this->a = a;
9          this->b = b;
10     }
11 };
12
13 int main() {
14     A *AA = new A();
15     AA = new A( a: 5, b: 78);
16     int *i = new int;
17     i = new int(10); //начальное значение
18     AA = new A[4]; //возможно только если задан конструктор по умолчанию
19     return 0;
20 }
```

Тк в c++ нет сборщика мусора нужно как-то освобождать память. Для этого существует оператор `delete`

```
i = new int(10); //начальное значение
A *AAA = new A[4]; //возможно только если задан конструктор по умолчанию

delete[] AAA;
delete i;
```

Для удаления массива нужно использовать квадратные скобки.

Билет 11.

Вопрос по Java: Экземплярные и статические вложенные классы. Локальные классы. Анонимные классы

Экземплярный вложенный класс – это класс объявленный внутри какого-то класса (класс внутри которого это объявляется называется объемлющим). Имеет доступ к внутреннему состоянию.

Статический вложенный класс – класс объявленный внутри объемлющего, имеет доступ к статическим полям объемлющему классу. В случае статического класса нужно писать `static`

Локальные классы – это классы объявленные внутри функции объемлющего класса. Имеет доступ к локальным неизменяемым переменным этой функции и её параметрам.

```
class A{
    int mag(){
        class loc{
            1 usage
            int lol;
            1 usage
            loc(int i){
                lol = i;
            }
        }
        loc l = new loc(i: 12);
        return 0;
    }
}
```

Анонимный класс – локальный класс, объявление которого совмещено с созданием его экземпляра

```
public class Main {
    public static void main(String[] args) {
        // Создание объекта интерфейса
        MyInterface obj = new MyInterface() {
            1 usage
            @Override
            public void myMethod() {
                System.out.println("Вызов метода из анонимного класса");
            }
        };

        // Вызов метода из анонимного класса
        obj.myMethod();
    }
}

// Интерфейс
2 usages 1 implementation
interface MyInterface {
    1 usage 1 implementation
    void myMethod();
}
```

Вопрос по C++: Объявление конструкторов.

Объявление конструктора в c++ является объявлением экземплярного метода без указания типа и с названием совпадающего с именем класса. Можно поставить модификатор private

перед конструктором и тогда мы запретим создание экземпляров класса. Пример:

```
#include <iostream>

class A{
public:
    int a, b;
    A(){}
    A(int a, int b){
        this->a = a;
        this->b = b;
    }
};
```

Билет 12.

Вопрос по Java: Функциональные интерфейсы и лямбда-выражения.

Функциональный интерфейс, это интерфейс с одним методом. Лямбда выражение может являться реализацией этого интерфейса. По сути (параметры) -> метод/выражение является записью создания класса с определённым внутренним состоянием и реализацией внутри него метода/выражения.

1. (форм. параметры) -> тело_метода	<table border="1"><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr></table>						
2. (форм. параметры) -> выражение							

```
1 interface IntBinaryExpr {
2     int eval(int x, int y);
3 }
4
5 public class Test {
6     public static void main(String[] args) {
7         IntBinaryExpr e = (x, y) -> x + y;
8
9         /* До Java 1.8 было бы:
10        IntBinaryExpr e = new IntBinaryExpr() {
11            public int eval(int x, int y) {
12                return x + y;
13            }
14        };
15        */
16
17        System.out.println(e.eval(10, 20));
18    }
19 }
```

Вопрос по C++: Объявление полей. Определения статических полей. Объявление методов. Виртуальные и абстрактные методы.

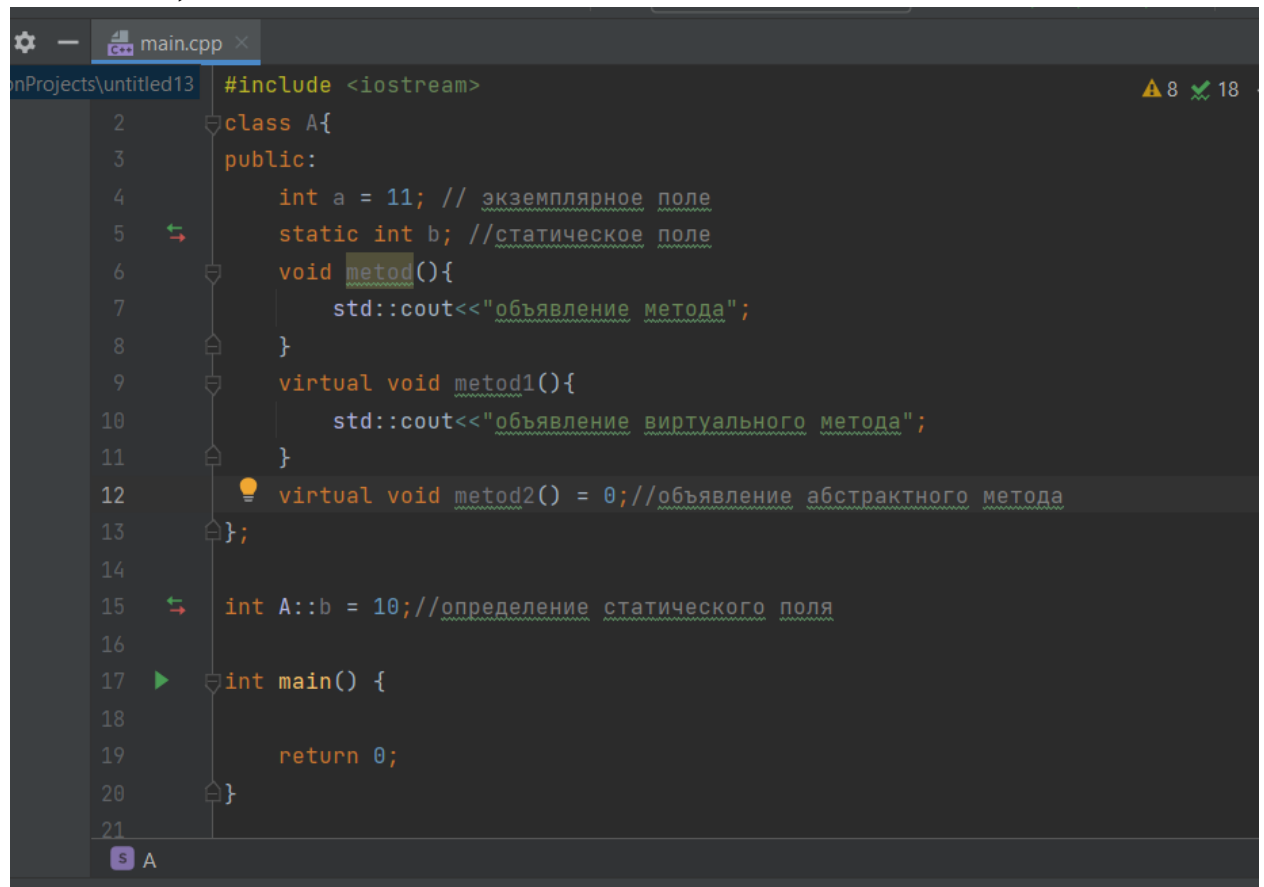
Поля объявляются внутри класса, могут иметь модификатор доступа. Статические поля так же объявляются внутри класса, но тк по факту это глобальные переменные определение их происходит за классом чаще всего .cpp используя прототип

тип ИмяКласса:ИмяПоля = значения;

Методы объявляются в теле класса, могут быть определены за его пределами.

Виртуальные методы – это методы которые могут быть переопределены в классе наследнике, объявляются ключевым словом `virtual`. Далее идёт обычный метод.

Абстрактный метод объявляется так же как виртуальный, только вместо тела метода пишется `= 0`;



```
#include <iostream>

class A{
public:
    int a = 11; // экземплярное поле
    static int b; //статическое поле
    void metod(){
        std::cout<<"объявление метода";
    }
    virtual void metod1(){
        std::cout<<"объявление виртуального метода";
    }
    virtual void metod2() = 0; //объявление абстрактного метода
};

int A::b = 10; //определение статического поля

int main() {
    return 0;
}
```

Билет 13.

Вопрос по Java: Необходимость обобщений. Контейнерные классы. Основная проблема при использовании необобщённых контейнерных классов.

Обобщения – это технология написания классов принимающие различные типы. Позволяет избежать дублирование кода и упростить поддержку и разработку программы.

Контейнерный класс – класс объекты которого являются хранилищем других типов.

Основная проблема связана с тем, что необходимо прописывать для каждого типа свои методы, конструкторы и тп.

Вопрос по C++: Объявление класса. Секции в объявлении класса.

Объявление класса: `class имя класса: суперклассы, если есть {тело класса};`

Секциями являются блоки кода, для которых применяется какой-то модификатор

- `public`: поля и методы, объявленные в этой секции, будут доступны из любого места программы.

- `private`: поля и методы, объявленные в этой секции, будут доступны только внутри класса.

- protected: поля и методы, объявленные в этой секции, будут доступны только внутри класса и его наследников.

По умолчанию, если не указана никакая секция, то используется private.

Билет 14.

Вопрос по Java: Понятие обобщённого класса. Ограниченные обобщённые классы.

Обобщенный класс – это класс для которого заданы формальные параметры после имени в треугольных скобках. class Имя<формальные параметры>

Формальный типовой параметр – это тип который используется в объявлении обобщенного класса и не известен во время компиляции обобщенного класса

Ограниченным он считается, если хотя бы для одного типового параметра есть ограничение.

Ограничение сверху (upper bound) ограничивает тип параметра типом, который находится выше в иерархии наследования. Например, если мы хотим ограничить параметр типа только классами, которые наследуются от класса Animal, мы можем написать следующее:

```
public class MyClass<T extends Animal> {  
    // ...  
}
```

Ограничение снизу (lower bound) ограничивает тип параметра типом, который находится ниже в иерархии наследования. Например, если мы хотим ограничить параметр типа только классами, которые являются суперклассами для класса Animal, мы можем написать следующее:

```
public class MyClass<T super Animal> {  
    // ...  
}
```

Вопрос по C++: Ссылки как возвращаемые значения функций. Константные ссылки.

Функция может возвращать ссылку на объект, тогда можно будет изменять объект за пределами функции. Не стоит ссылаться на локальные переменные. Константные ссылки – это ссылки по которым нельзя изменить переменную.

Билет 15.

Вопрос по Java: Ковариантность массивов. Инвариантность обобщённых классов. Шаблоны обобщённых классов. Особенности шаблонов.

В java массивы ковариантны. Если класс А наследник класса Б, то A[] подтип B[]

```
Integer[] ints = new Integer[10];  
Number[] numbers = ints;
```

// OK

Обобщенные классы инвариативны – если А наследует В, то S<А> не связаны наследованием S<В>.

Шаблон обобщённого класса G – это неявный супертип для любого класса, порождённого из G. Шаблон получается параметризацией класса G специальным фактическим параметром «?». Например, Stack<?>.

Особенности в том, что можно создать переменную типа шаблона Stack<T> например, но нельзя создать его объект.

Если тип переменной – шаблон некоторого обобщенного класса G, то этой переменной можно присвоить ссылку на любой объект класса G, независимо от того, какой фактический типовый параметр был передан классу G при создании объекта

Вопрос по C++: Объявление переменных ссылочного типа. Инициализация и использование ссылок. Ссылки в формальных параметрах функций.

Переменная ссылочного типа объявляется как:

тип данных & имя переменной

Ссылка в C++ – это типизированный указатель, к которому неприменимы арифметические операции, и который не может быть нулевым. Кроме того, недопустимы ссылки на ссылки

Ссылка – указатель

Не может изменяться арифметически

Не нулевой

Ссылок на ссылки нет

При этом ссылки в глобальных и локальных переменных должны быть обязательно инициализированы при объявлении, ссылки в полях объектов должны быть инициализированы в конструкторе класса. Более того, значение, полученное ссылкой при инициализации, в дальнейшем не может быть изменено

По сути штука которую надо задать сразу и она останется навсегда. Инициализировать ссылку можно просто написать = после объявления и указать переменную.

Например

```
int &a = b;
```

Использовать ссылку можно как обычную переменную, изменяться будет переменная на которую указывает ссылка.

В формальных параметрах может быть указана ссылка. Тогда она будет инициализированная переменной которую мы передаем в функцию, те указывать на неё.

Билет 16.

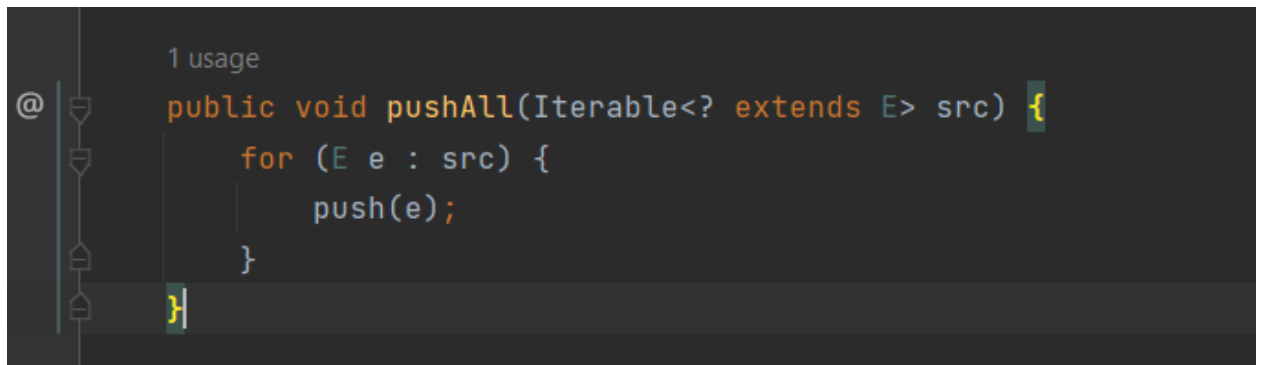
Вопрос по Java: Полиморфизм на основе шаблонов. Частичные шаблоны. Шаблоны, ограниченные сверху. Методы, недоступные у шаблонов, ограниченных сверху.

Полиморфизм в ооп – это способность кода обрабатывать код разных типов. На основе шаблонов можно получить хороший, настоящий полиморфизм.

Если обобщённый класс имеет несколько формальных типовых параметров, то возможно получить частичный шаблон, передав классу «?»(wildcard) вместо части фактических типовых параметров.

Ограничение сверху (upper bound) ограничивает тип параметра типом, который находится выше в иерархии наследования.

`<? extends Class>`



```
1 usage
@
public void pushAll(Iterable<? extends E> src) {
    for (E e : src) {
        push(e);
    }
}
```

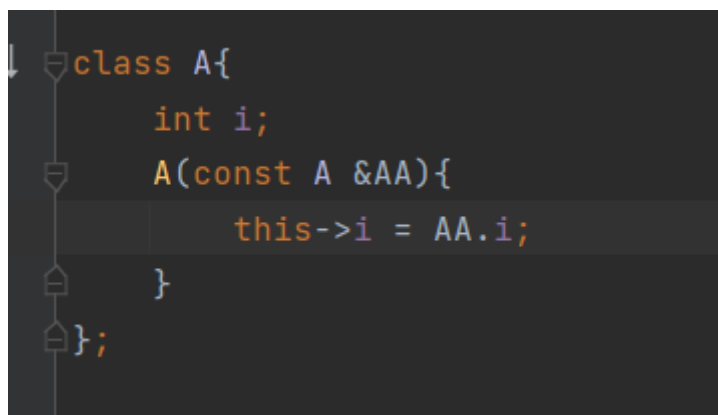
Недоступны методы для которых нужны типы которые попали под ограничение.

Например, если мы ограничили `<? extends Number>`, то не будут работать методы которые предполагали передачу строкового значения.

Вопрос по C++: Проблема копирования объектов. Объявление конструктора копий. Перегруженная операция присваивания.

Проблема копирования – по умолчанию создается побитовая копия внутреннего состояния объекта. Но есть ситуации когда это не работает хорошо, например у при копировании массива у нас будет указатель на один и тот же массив, что при вызове деструктора одного из объектов приведет к потере массива.

Для правильного копирования создается конструктор копий. По сути обычный конструктор в который мы передаем константный объект этого класса. Например:



```
class A{
    int i;
    A(const A &AA){
        this->i = AA.i;
    }
};
```

Перегрузка операции присваивания, перегружаем операцию присваивания правильным образом, чтоб создавалась копия

```

    }
    A& operator=(const A &AA){
        this->i = AA.i;
    }
};

```

В примерах не решается проблема копирования, тк было лениво её решать. Но вы можете без проблем додумать сами её решение. На всякий случай скину из других источников фото

Этот конструктор будет автоматически вызываться всякий раз, когда необходимо создать копию объекта.

```

IntArray::IntArray(const IntArray &obj): n(obj.n) {
    a = new int [n];
    std::copy(obj.a, obj.a + n, a);
}

```

нужно

Также необходимо переопределить операцию присваивания

```

IntArray& IntArray::operator= (const IntArray &obj) {
    if (this != &obj) {
        int *new_a = new int [obj.n];
        std::copy(obj.a, obj.a + obj.n, new_a);
        delete [] a;
        n = obj.n;
        a = new_a;
    }
    return *this;
}

```

нужно

Билет 17.

Вопрос по Java: Шаблоны, ограниченные снизу. Методы, недоступные у шаблонов, ограниченных снизу

Ограничение снизу (lower bound) ограничивает тип параметра типом, который находится ниже в иерархии наследования.

Запись <? super Class>

```

3
4      1 usage
5      public void popAll(Collection<? super E> out) {
6          while (!isEmpty()) {
7              out.add(pop());
8          }
9      }

```

Недоступны методы для которых нужны типы которые попали под ограничение. Например, если мы ограничили `<? super Number>`, то не будут работать методы которые предполагали передачу типов наследуемых у `Number` (например `int`).

Вопрос по C++: Ссылки как возвращаемые значения функций. Константные ссылки.

Функция может возвращать ссылку на объект, тогда можно будет изменять объект за пределами функции. Не стоит ссылаться на локальные переменные. Константные ссылки – это ссылки по которым нельзя изменить переменную.

Билет 18.

Вопрос по Java: Типовые параметры у методов. Объявление и вызов обобщённого метода.

В джаве могут быть типовые параметры у методов

Обобщённый метод – это статический или экземплярный метод класса, имеющий формальные типовые параметры. В объявлении метода типовые параметры непосредственно предшествуют типу возвращаемого значения метода

A screenshot of a code editor showing a generic method in Java. The code is:

```
public <T> T t(T t){  
    return t;  
}
```

 The text is highlighted in a dark theme with syntax coloring.

Вопрос по C++: Объявление класса. Секции в объявлении класса.

Объявление класса: `class имя класса: суперклассы, если есть {тело класса};`

Секциями являются блоки кода, для которых применяется какой-то модификатор

- `public`: поля и методы, объявленные в этой секции, будут доступны из любого места программы.
- `private`: поля и методы, объявленные в этой секции, будут доступны только внутри класса.
- `protected`: поля и методы, объявленные в этой секции, будут доступны только внутри класса и его наследников.

По умолчанию, если не указана никакая секция, то используется `private`.

Билет 19.

Вопрос по Java: Нештатные ситуации. Перехват нештатных ситуаций.

Нештатная ситуация – это ситуация, в которой выполнение некоторого фрагмента кода программы оказывается по тем или иным причинам невозможно.

Исключительная ситуация – это нештатная ситуация, возникшая в силу внешних по отношению к программе причин.(например передали значения и они не были предусмотрены, либо специально вызвали ошибку странными действиями)

Ошибочная ситуация – это нештатная ситуация, возникшая из-за ошибки в коде программы. (ошиблись в коде)

Перехват нештатных ситуации – это механизм, обеспечивающий продолжение работы программы при возникновении нештатной ситуации.

Функция, во время выполнения которой может возникнуть нештатная ситуация, возвращает некоторое значение, которое говорит о том, успешно или неуспешно функция выполнила свою задачу. Перехват ситуации заключается в том, что в коде, вызывающем такую функцию, стоят проверки ее возвращаемого значения

В случае возникновения нештатной ситуации генерируется так называемое исключение, описывающее нештатную ситуацию. Генерация исключения приводит к передаче управления на фрагмент кода программы, называемый обработчиком исключения

Вопрос по C++: Объявление переменных ссылочного типа. Инициализация и использование ссылок. Ссылки в формальных параметрах функций.

Переменная ссылочного типа объявляется как:

тип данных & имя переменной

Ссылка в C++ – это типизированный указатель, к которому неприменимы арифметические операции, и который не может быть нулевым. Кроме того, недопустимы ссылки на ссылки

Ссылка – указатель

Не может изменяться арифметически

Не нулевой

Ссылок на ссылки нет

При этом ссылки в глобальных и локальных переменных должны быть обязательно инициализированы при объявлении, ссылки в полях объектов должны быть инициализированы в конструкторе класса. Более того, значение, полученное ссылкой при инициализации, в дальнейшем не может быть изменено

По сути штука которую надо задать сразу и она останется навсегда. Инициализировать ссылку можно просто написать = после объявления и указать переменную.

Например

```
int &a = b;
```

Использовать ссылку можно как обычную переменную, изменяться будет переменная на которую указывает ссылка.

В формальных параметрах может быть указана ссылка. Тогда она будет инициализированная переменной которую мы передаем в функцию, те указывать на неё.

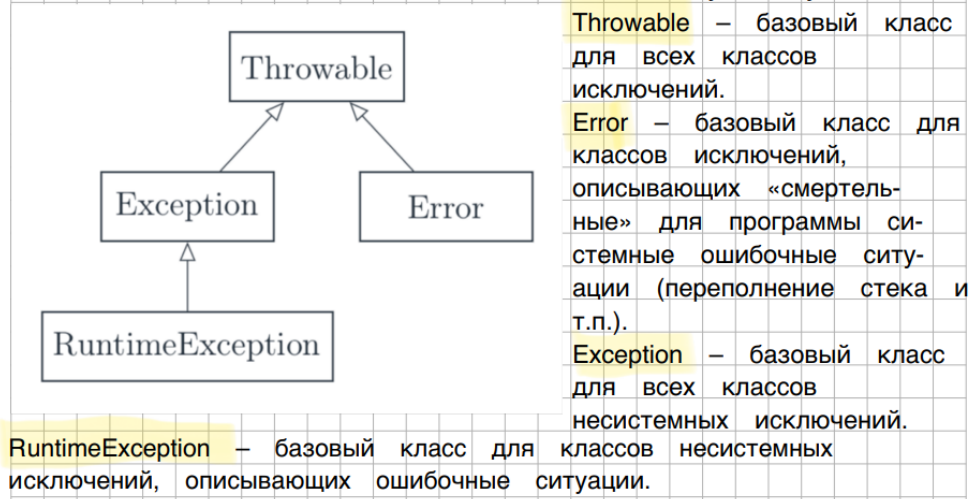
Вопрос по Java: Иерархия классов исключений. Методы класса Throwable.

Throwable – базовый класс всех исключений

Error – базовый класс для исключений описывающих смертельные для программы системные ошибочные ситуации

Exception – базовый класс для всех несистемных исключений. RuntimeException – базовый класс для классов несистемных исключений, описывающих ошибочные ситуации

Исключение – это объект, описывающий нештатную ситуацию.



Методы класса Throwable

Конструкторы:

Throwable (String message)

Throwable (String message, Throwable cause)

Throwable getCause()

String getMessage()

String toString()

void printStackTrace()

Вопрос по C++: Объявление полей. Определения статических полей. Объявление методов. Виртуальные и абстрактные методы.

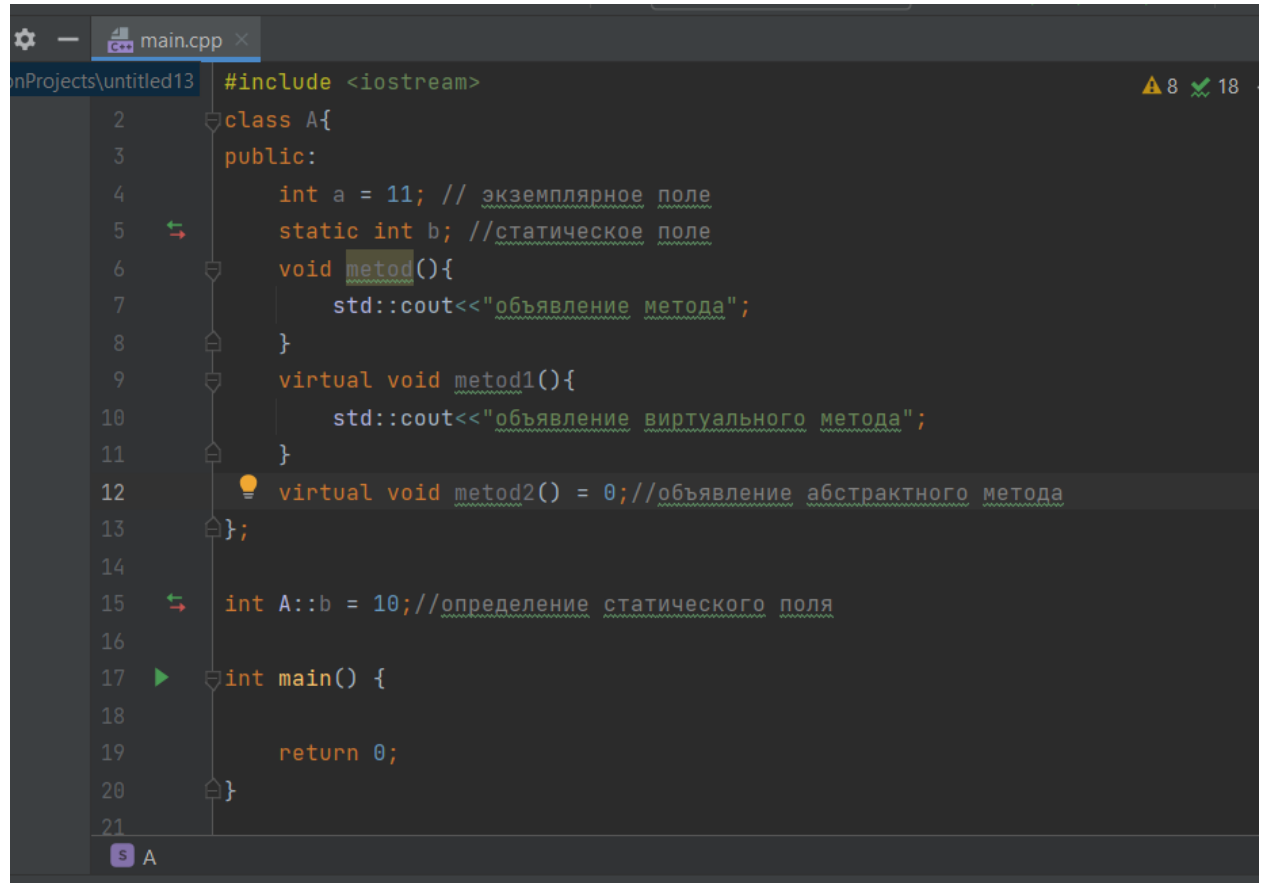
Поля объявляются внутри класса, могут иметь модификатор доступа. Статические поля так же объявляются внутри класса, но так по факту это глобальные переменные, определение их происходит за классом чаще всего .cpp используя прототип

тип ИмяКласса::ИмяПоля = значения;

Методы объявляются в теле класса, могут быть определены за его пределами.

Виртуальные методы – это методы которые могут быть переопределены в классе наследнике, объявляются ключевым словом virtual. Далее идет обычный метод.

Абстрактный метод объявляется так же как виртуальный, только вместо тела метода пишется = 0;



```
#include <iostream>

class A{
public:
    int a = 11; // экземплярное поле
    static int b; //статическое поле
    void metod(){
        std::cout<<"объявление метода";
    }
    virtual void metod1(){
        std::cout<<"объявление виртуального метода";
    }
    virtual void metod2() = 0; //объявление абстрактного метода
};

int A::b = 10; //определение статического поля

int main() {
    return 0;
}
```

Билет 21.

Вопрос по Java: Неперехваченные исключения. Операторы перехвата исключений.

Неперехваченные исключения приводят к аварийному завершению программы с выдачей стеков вызовов, записанных в исключения.

Перехват исключений проходит посредством оператора try catch

Пример использования блока finally:

```
try {
    // блок кода, который может вызвать исключение
} catch (Exception e) {
    // блок кода, который обрабатывает исключение
} finally {
    // блок кода, который будет выполнен независимо от того, было ли исключение перехвачено или нет
}
```

Вопрос по C++: Создание объектов в динамической памяти. Создание массивов в динамической памяти. Удаление объектов и массивов.

Динамическое выделение памяти под значение любого типа выполняется в C++ операцией new. Она совмещает выделение памяти с инициализацией (с вызовом конструктора) и возвращает указатель на созданный и инициализированный объект. Примеры:

```

1      #include <iostream>
2      class A{
3      public:
4          int a, b;
5          A(){}
6          A(int a, int b){
7              this->a = a;
8              this->b = b;
9          }
10     };
11
12     int main() {
13         A *AA = new A();
14         AA = new A( a: 5, b: 78);
15         int *i = new int;
16         i = new int(10); //начальное значение
17         AA= new A[4]; //возможно только если задан конструктор по умолчанию
18         return 0;
19     }
20

```

Тк в с++ нет сборщика мусора нужно как-то освобождать память. Для этого существует оператор delete

```

i = new int(10); //начальное значение
A *AAA = new A[4]; //возможно только если задан конструктор по умолчанию

delete[] AAA;
delete i;

```

Для удаления массива нужно использовать квадратные скобки.

Билет 22.

Вопрос по Java: Порождение исключения

Порождение исключения выполняется оператором throw исключение; Например, throw new SomeException ()

Если класс исключения не является подклассом классов Error и RuntimeException, то для любого порождённого исключения в коде программы должен быть предусмотрен обработчик. Это требование синтаксически выражается следующим образом: если в процессе выполнения некоторого метода m могут порождаться исключения x, y и z, и эти исключения в теле метода не перехватываются, то объявление метода должно выглядеть как тип m(параметры) throws x, y, z { ... }(что-то вроде игнорирования ошибок, чтоб аварийно не выйти как я понимаю)

Вопрос по C++: Объявление конструкторов.

Объявление конструктора в с++ является объявлением экземплярного метода без указания типа и с названием совпадающего с именем класса. Можно поставить модификатор private

перед конструктором и тогда мы запретим создание экземпляров класса. Пример:

```
#include <iostream>

class A{
public:
    int a, b;
    A(){}
    A(int a, int b){
        this->a = a;
        this->b = b;
    }
};
```

Билет 23.

**Вопрос по Java: Понятие функтора. Реализация метода map в функторе Roots.
Перебор корней в функторе Roots. Использование функтора Roots.**

Функтор – это обобщённый класс, параметризованный типом T и инкапсулирующий значение(-я) типа T, удовлетворяющий следующим условиям

1. Значения в функторе не меняются
2. Есть метод map который принимает функцию и применяет её к объектам функтора
3. Метод map создает новый объект функтора в который инкапсулирует туда результат
4. При передачи в map функции которая возвращает своё же значение должен быть возвращен объект схожий с текущим функтором

2 usages

```
public class Roots<T> {
```

2 usages

```
    HashSet<T> cont;
```

1 usage

```
    Roots(HashSet<T> H){
```

```
        cont = H;
```

```
    }
```

```
    public <R> Roots<R> map(Function<T, R> f){
```

```
        HashSet<R> c = new HashSet<>();
```

```
        for(T C : cont){
```

```
            c.add(f.apply(C));
```

```
        }
```

```
        return new Roots<R>(c);
```

```
    }
```

```
}
```

import

```
java.util.*;
```

```
import java.util.function.Consumer;
```

```
import java.util.function.Function;
```

```
public class Roots<T> {
```

```
    HashSet<T> cont;
```

```
    Roots(HashSet<T> H){
```

```
        cont = H;
```

```
    }
```

```
    Roots(){
```

```
        cont = new HashSet<>();
```

```
    }
```

```
    public void add(T t){
```

```
        cont.add(t);
```

```
    }
```

```
    public void forEach(Consumer<T> f){
```

```
        for(T C : cont){
```

```
            f.accept(C);
```

```
        }
```

```
    }
```

```
    public <R> Roots<R> map(Function<T, R> f){
```

```
        HashSet<R> c = new HashSet<>();
```

```
        for(T C : cont){
```

```
            c.add(f.apply(C));
```

```
        }
```

```
        return new Roots<R>(c);
```

```
    }
```

```

public static void main(String[] args){
    Roots<Integer> R = new Roots<Integer>();
    R.add(10);
    R.add(12);
    R.add(5);
    R = R.map(x->x*10);
    R.forEach(System.out::println);
}
}

```

Вопрос по C++: Объявление деструктора. Объекты в автоматической памяти. Автоматический вызов деструктора.

Деструктор – это экземплярный метод, предназначенный для освобождения ресурсов, принадлежащих объекту, непосредственно перед уничтожением этого объекта.

Пример:

```

~A(){
}

```

В классе могут быть ссылки и различные данные которые при удалении объекта приводят к утечке данных. В языках без сборки мусора нужно прописывать деструктор, который освобождает ресурсы.

Автоматическая переменная (она же переменная в автоматической памяти и только локальные переменные могут быть автоматическими). Память под автоматическую переменную выделяется автоматически всякий раз, когда исполнение кода достигает определения этой переменной

Объекты в автоматической памяти – создаются внутри блока кода и когда блок кода завершается автоматически вызывается деструктор этих объектов.

Пример таких объектов

```

17
18 void F(){
19     int a;
20     int b;
21     A AA;
22 }
23

```

Билет 24.

Вопрос по Java: Введение в монады на примере класса Roots. Добавление метода flatMap в класс Roots. Использование метода flatMap.

Если у нас начинают вкладываться друг в друга типы, например при решении уравнения 4й степени

Что, если мы хотим использовать функтор **Roots** для хранения корней уравнения четвёртой степени вида

$$a(x^2 + px)^2 + b(x^2 + px) + c = 0?$$

Очевидно, что уравнение имеет до четырёх корней типа **double**, которые было бы удобно хранить в объекте типа **Roots<Double>**, а не **Roots<Roots<Double>>**.

Избавиться от обозначенной на предыдущем слайде проблемы можно, превратив функтор **Roots** в *монаду* путём добавления метода **flatMap**:

```
import java.util.*;
import java.util.function.Consumer;
import java.util.function.Function;

public class Roots<T> {
    HashSet<T> cont;
    Roots(HashSet<T> H){
        cont = H;
    }
    Roots(){
        cont = new HashSet<>();
    }
    public void add(T t){
        cont.add(t);
    }

    public void forEach(Consumer<T> f){
        for(T C : cont){
            f.accept(C);
        }
    }

    public <R> Roots<R> flatMap(Function<T, Roots<R>> f){
        HashSet<R> c = new HashSet<>();
        map(f).forEach(r -> c.addAll(r.cont));
        return new Roots<R>(c);
    }
    public <R> Roots<R> map(Function<T, R> f){
        HashSet<R> c = new HashSet<>();
        for(T C : cont){
            c.add(f.apply(C));
        }
        return new Roots<R>(c);
    }

    public static void main(String[] args){
        Roots<Integer> R = new Roots<Integer>();
        R.add(10);
        R.add(12);
        R.add(5);
        R = R.map(x->x*10);
        R.forEach(System.out::println);
    }
}
```

Метод **flatMap** «уплощает» корни, объединяя множества корней из нескольких объектов класса **Roots** в одно множество.

Теперь мы можем получить сразу все корни нашего уравнения четвёртой степени в виде **Roots<Double>**:

```
Roots<Double> roots =
    Roots.of(a, b, c, 1e-10)
        .flatMap(y -> Roots.of(1.0, p, -y, 1e-10));
```

Вопрос по C++: Объекты в глобальной памяти. Объекты в полях других объектов.

Если объект размещается в глобальной переменной или в статическом поле класса, то конструктор для него вызывается до передачи управления в функцию `main`, а деструктор – после завершения функции `main`.

Если объект находится в теле другого класса, то его конструктор можно вызвать из конструктора другого класса таким образом:

```
class B{
public:
    B(int ballIU9){
        std::cout<<ballIU9;
    }
};

class A{
    B BB;
    A(int i) : BB( ballIU9: i){}
};
```

Если мы захотим в данном примере удалить класс `A`, то автоматически вызовется деструктор и объекта класса `B`

Билет 25.

Вопрос по Java: Определение монады. Проблема нулевых ссылок. «Заворачивание» значения в `Optional`. «Разворачивание» `Optional`. Композиция частичных функций.

Монада – это функтор с дополнительно методом `flatMap`, который:

1. Получает в качестве параметра функцию, способную преобразовать каждый элемент и завернуть результат преобразования в новый объект монаду
2. Применяет функцию ко всем элементам
3. Ассоциативен

• метод `flatMap` ассоциативен, т.е. выражение `m.flatMap(f).flatMap(g)` эквивалентно выражению `m.flatMap(x -> f(x).flatMap(g))`

Проблема нулевых ссылок – индикатором отсутствия значений в `java` является нулевая ссылка, в случае когда программист забывает проверить значение на нулевую ссылку можно легко поймать ошибку `NullPointerException`

Завернуть значение в опционал можно 3 способами

1. Создать пустой опционал

}

```

class A{
    public:
        int x;
        A(int x){
            this->x = x;
        }
};

class B : public A{
    int y;
    B(int y, int x) : A(x){
        this->y = y;
    }
};

```

Билет 26.

Вопрос по Java: Метод map. Композиция частичных функций с использованием Optional.

Метод класса Опционал позволяет обойтись без проверок при записи композиции частичных сумм. Метод принимает лямбда функцию определяющую частичную функцию

Метод мап ничего не делает если опционал пуст и возвращает пустой опционал, иначе применяет лямбду и возвращает новую опционал

По сути просто применяем все функции от а до с последовательно

```

public static Optional<String> compose(
    HashMap<String, Integer> a,
    HashMap<Integer, Float> b,
    HashMap<Float, String> c,
    String x)
{
    return Optional.ofNullable(x)
        .map(a::get)
        .map(b::get)
        .map(c::get);
}

```

Вопрос по C++: Объявление полей. Определения статических полей. Объявление методов. Виртуальные и абстрактные методы.

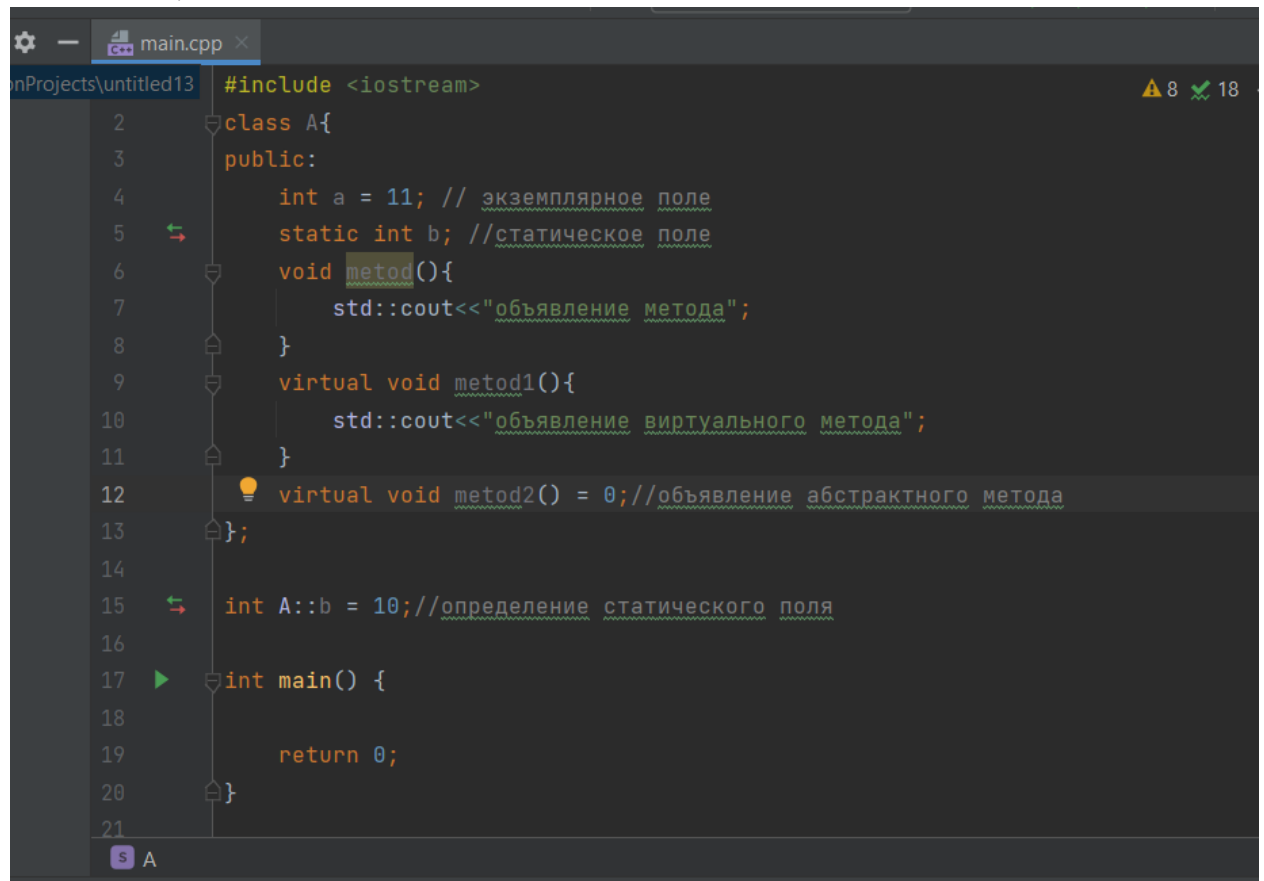
Поля объявляются внутри класса, могут иметь модификатор доступа. Статические поля так же объявляются внутри класса, но тк по факту это глобальные переменные определение их происходит за классом чаще всего .cpp используя прототип

тип ИмяКласса::ИмяПоля = значения;

Методы объявляются в теле класса, могут быть определены за его пределами.

Виртуальные методы – это методы которые могут быть переопределены в классе наследнике, объявляются ключевым словом `virtual`. Далее идёт обычный метод.

Абстрактный метод объявляется так же как виртуальный, только вместо тела метода пишется `= 0`;



```
#include <iostream>

class A{
public:
    int a = 11; // экземплярное поле
    static int b; //статическое поле
    void metod(){
        std::cout<<"объявление метода";
    }
    virtual void metod1(){
        std::cout<<"объявление виртуального метода";
    }
    virtual void metod2() = 0; //объявление абстрактного метода
};

int A::b = 10; //определение статического поля

int main() {
    return 0;
}
```

Билет 27.

Вопрос по Java: Потоки как развитие идеи итераторов. Преобразование последовательностей. Filter и map, flatMap. Собирающие последовательности

127	Вопрос по Java: Потоки как развитие идеи итераторов. Преобразование последовательностей. Filter и map, flatMap. Собирающие последовательности.
	Интерфейс Stream<T> в каком-то смысле является развитием идеи итераторов, т.е. представляет последовательность некоторых значений, называемую потоком.
	Получить объект Stream<T> можно из объекта любого контейнерного класса стандартной библиотеки языка Java путём вызова метода stream: Stream<T> stream()
	Вытащить все значения из потока Stream<T> можно с помощью метода forEach: void forEach(Consumer<? super T> action) Этот метод получает в качестве параметра лямбда-выражение и вызывает его для каждого значения из последовательности, представляемой потоком.
	Методы интерфейса Stream<T> позволяющие преобразовывать последовательности:
	● формирование потока, содержащего только те элементы исходного потока, которые удовлетворяют предикату: Stream<T> filter(Predicate<? super T> predicate) ● формирование потока, представляющего последовательность значений, получаемую путём вызова функции для каждого элемента исходного потока: <R> Stream<R> map(Function<? super T, ? extends R> mapper) ● конкатенация потоков, получаемых путём вызова функции для каждого элемента исходного потока: <R> Stream<R> flatMap(Function<? super T, ? extends Stream<? extends R>> mapper)
	Потребление значений из потоков выполняется так называемыми коллекторами – объектами классов, реализующими интерфейс Collector. Коллекторы предназначены для выполнения анализа потоков и записи результатов реализуемых потоками преобразований в объекты контейнерных классов.
	Чтобы задействовать некоторый коллектор, следует вызвать метод collect потока: <R,A> R collect(Collector<? super T,A,R> collector)

Вопрос по C++: Наследование от нескольких базовых классов. Противоречия в именах наследуемых членов класса. Разрешение противоречий в именах.

C++ поддерживает множественное наследование. Пусть класс A наследует классы AA, AAA, и AAAA, тогда это будет записываться как:

```
class A : AA, AAA, AAAA{}
```

Также есть возможность установить модификатор доступа для наследуемых классов, например:

```
class A : public AA, AAA, AAAA{}
```

При множественном наследовании может возникнуть проблема одинаковых имен в различных базовых классах. Такая проблема разрешается конструкцией, когда мы

обращаемся к конкретному базовому классу от объекта. Например:

```
int main() {  
    C CC;  
    std::cout<<CC.A::x;  
    return 0;  
}
```

Билет 28.

Вопрос по Java: Механизм пакетов. Создание пакета. Правила видимости для классов пакета и их членов.

Начиная с определённого размера программной системы, для её структурирования начинает не хватать одной только декомпозиции на классы и требуется механизм пакетов: образуются группы взаимосвязанных классов, которые целесообразно как-то выделить в отдельные подсистемы и ограничить доступность некоторых классов рамками соответствующей подсистемы; возникает потребность упростить навигацию по исходным текстам программной системы (чтобы легче находить классы, которые по смыслу относятся к той или иной подсистеме); появляются трудности с изобретением уникальных имён классов (особенно в случае командной разработки). Пакет – это контейнер классов, предоставляющий для них отдельное пространство имён и дополнительные возможности по управлению доступом. Для создания пакета нужно поместить в начало каждого файла, содержащего исходный текст классов, которые вы планируете включить в проект, директиву: `package имя_пакета;` Файлы, где нет директивы `package`, в пакете по умолчанию

Публичные классы можно использовать как внутри, так и снаружи пакета, а непубличные классы доступны из любого места внутри пакета, но извне не видны. К членам классов пакета, объявленным без модификаторов доступа, можно обращаться из любого места внутри пакета, но запрещено обращаться снаружи пакета

Вопрос по C++: Иерархия наследования и классы противоречия. Основная проблема противоречивых иерархий.

Иерархия наследования – это ориентированный ациклический (направленный и без циклов) граф, в котором если класс А базовый и В наследует его, то от А к В проводится дуга.

Класс противоречия – это класс в который из какого-то другого класса ведут как минимум 2 пути. Пример, тут класс A будет классом противоречия

```
2
3  class A{
4      public:
5          int x = 10;
6      virtual int f(){
7          return 5;
8      }
9  };
10
11 class B : public A{
12     public:
13     int f(){
14         return 6;
15     }
16 };
17
18 class C : public A, B{
19 };
20
21
```

Основной проблемой класса противоречий является многократное включение полей и методов базового класса в произвольный.

Билет 29.

Вопрос по Java: Импорт классов из другого пакета. Импорт статических членов класса

Для того чтобы обратиться к классу, принадлежащему другому пакету, этот класс нужно импортировать. Существует три способа импорта класса.

1. Использование квалифицированного имени. Например, `java.util.HashSet a = new java.util.HashSet <>();`
2. Указание класса в директиве `import`: `import java.util.HashSet; // в начале файла ... HashSet a = new HashSet <>();`
3. Использование директивы `import` для импорта всех классов пакета: `import java.util.*; // в начале файла ... HashSet a = new HashSet <>();`

Для упрощения доступа к статическим членам классов существует специальная форма директивы `import`, позволяющая импортировать их в текущий файл. После импорта пишется слово `static`.

Вопрос по C++: Динамическое приведение типов.

Для динамического приведения в c++ используется оператор `dynamic_cast` он принимает на вход ссылку на объект и тип необходимой ссылки `T`. Если он находит в объекте тип `T` то возвращает ссылку на него, иначе нулевую ссылку.

```
|
|
▶ int main() {
    B BB;
    A *AA = &BB;
    B *BBB = dynamic_cast<B*>(AA);
    std::cout<<BBB->f();
    return 0;
}
```